

ENTORNOS DE DESARROLLO

Reto 2: POO en Python

Alumno: Francisco José García Franco

1º DAM - UAX

03/05/2026

0. Índice

1. Enunciado	1
2. Código en Python	2
3. Ejecución	4
4. Archivos adjuntos	5

1. Enunciado

Reto 2

Apertura: miércoles, 11 de marzo de 2026, 00:00

Cierre: sábado, 2 de mayo de 2026, 23:59

Se debe crear una clase llamada Personaje que represente a un personaje dentro de un videojuego.

La clase debe tener un constructor con tres atributos:

peso (entero)

altura (entero)

energía (entero)

El atributo energía deberá inicializarse siempre con el valor 100 al crear el personaje.

A continuación, se deberán implementar dos métodos:

Método recibirDanio()

Este método reducirá la energía del personaje según el valor indicado como parámetro.

Si la energía llega a 0 o menos, el programa deberá indicar que el personaje ha sido derrotado.

Método recargarEnergia()

Este método aumentará la energía del personaje según el valor indicado.

La energía no podrá superar nunca el valor máximo de 100.

Finalmente, en el programa principal se deberá:

Crear un objeto de la clase Personaje.

Simular una situación de juego donde el personaje:

recibe daño

recupera energía

Mostrar por pantalla los cambios en la energía del personaje.

Criterio	Descripción	Puntuación
Definición de la clase	La clase está correctamente definida con los atributos solicitados	2 puntos
Constructor	Inicializa correctamente los atributos y establece la energía en 100	2 puntos
Método de daño	Reduce correctamente la energía y detecta cuando llega a 0	2 puntos
Método de recuperación	Aumenta la energía sin superar el máximo establecido	2 puntos
Creación y uso del objeto	Se crea el objeto y se utilizan los métodos correctamente	1 punto
Claridad del código	Código ordenado, legible y con nombres adecuados	1 punto

2. Código en Python

```
# Alumno: Francisco José García Franco
# Asignatura: Entornos de Desarrollo
# Módulo: 1º DAM UAX
# Archivo: Reto2.py

# Definición de la clase
class Personaje:

    # Constructor con atributos nombre, peso, altura
    def __init__(self, nombre, peso, altura):
        self.nombre = nombre # String
        self.peso = peso # int
        self.altura = altura # int
        self.energia = 100 # int

    # Información por pantalla
    print(f"Se ha creado el personaje {self.nombre} con peso {self.peso} y altura {self.altura}")
    print(f"El nivel de energía de {self.nombre} es de {self.energia}/100\n")

    # Método de reducción de energía por daño
    def recibirDanio(self, danio):

        self.energia -= danio
        print(f"{self.nombre} ha recibido un daño")

        if self.energia <= 0:
            print(f"¡¡¡DAÑO FATAL!!!: {self.nombre} ha sido derrotado\n")
            exit() # Sale del programa
        else:
            print(f"El nivel de energía de {self.nombre} es de {self.energia}/100\n")

    # Método de recarga de energía
    def recargarEnergia(self, incremento):

        self.energia += incremento
        print(f"{self.nombre} ha recargado su nivel de energía")

        if self.energia > 100:
            self.energia = 100
            print(f"El nivel de energía de {self.nombre} es de {self.energia}/100\n")

if __name__ == "__main__":

    print("CREACIÓN Y TRATAMIENTO DE ENERGÍA DE PERSONAJE")
    print("#####\n")

    # Creación de personaje
    personaje1 = Personaje("Vladimir", 78, 180)

    # Simulación de pérdida de energía y recarga
    personaje1.recibirDanio(10)
    personaje1.recargarEnergia(15)

    # Simulación de derrota del personaje
    personaje1.recibirDanio(150)
```

> Al crear la clase se ha considerado añadir un atributo **nombre** (no solicitado en el enunciado) para identificar más fácilmente al personaje.

> Cada método implementa mensajes por pantalla mostrando información del resultado de la operación realizada (**aumento** o **disminución** de **energía**) y la afección al personaje. En caso de pérdida total de energía se indica la **derrota** del mismo.

> Se crea un punto de ejecución principal "**if __name__ == '__main__':**" para crear un personaje y probar los métodos:

- > Se realiza una **primera simulación** de pérdida de energía sin derrota del personaje y posterior recarga.

- > Se realiza una **segunda simulación** que implica la derrota del personaje, terminando el programa con un `exit()`

3. Ejecución

```
C:\Users\fcojg\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\fcojg\PycharmProjects\PythonProject\Reto2.py
CREACIÓN Y TRATAMIENTO DE ENERGÍA DE PERSONAJE
#####

Se ha creado el personaje Vladimir con peso 78 y altura 180
El nivel de energía de Vladimir es de 100/100

Vladimir ha recibido un daño
El nivel de energía de Vladimir es de 90/100

Vladimir ha recargado su nivel de energía
El nivel de energía de Vladimir es de 100/100

Vladimir ha recibido un daño
¡¡¡DAÑO FATAL!!!: Vladimir ha sido derrotado

Process finished with exit code 0
```

4. Archivos adjuntos

> Se adjunta una carpeta comprimida **"Reto2-FranciscoJoseGarciaFranco.zip"** que contiene:

> El archivo **"Reto2.py"** con el código Python.

Nombre	Fecha de modificación	Tipo	Tamaño
 Reto2.py	03/05/2026 14:15	Python File	2 KB